

Machine Learning Project Report

Task 2 : Feature Engineering, Model Optimization & Performance Comparison

Prepared By: GOPAL
21st May, 2026

1. Methodology and Objectives

This report presents the implementation, optimization, and evaluation pipelines built for the house price prediction task using the California Housing Dataset. The primary goal is to establish a rigorous framework that moves beyond simple model instantiation to professional-grade iterative development. The target variable is the median house value (**HousePrice**), modeled using a collection of key regional baseline metrics.

The engineering pipeline was structured into five distinct operational phases:

- **Data Integration & Inspection:** Programmatic extraction of features and labels from the source framework, merging them into unified structures for schema confirmation.
- **Feature Isolation:** Structural decoupling of target vectors (**y**) from the baseline input matrix (**X**).
- **Data Standardization:** Normalization of distinct numerical features to eliminate scaling bias across mismatched bounds.
- **Validation Isolation:** Dividing datasets into distinct training and test sets using fixed seeding to ensure exact experimental replication.
- **Multi-Algorithm Iteration:** Parallel processing and tracking of three distinct algorithmic models over performance matrices.

2. Technical Execution and Scaling

Machine learning models require uniform input features to ensure unbiased weight updates. Without proper scaling, variables with high absolute magnitudes (such as population tracking arrays) would drown out critical minor variance metrics (such as average occupancy metrics). To resolve this, a mathematical transformation was executed across the input array using **StandardScaler**. This operation recalculates every individual variable configuration to fit a zero-centered distribution profile, governed by the following formula:

$$z = (x - \mu) / \sigma$$

Where μ represents the statistical mean of the target attribute feature, and σ represents the absolute standard deviation. Once standardized, the data array was cleanly partitioned into an 80% allocation for active parameter updates and a remaining 20% validation pool reserved exclusively for post-training test evaluations.

3. Baseline and Optimized Model Configurations

To establish a comparative baseline framework, three distinct modeling strategies were initiated simultaneously:

- **Linear Regression:** Implemented as a rigid, low-variance foundational benchmark assuming fixed linear relationships between coordinates.

- **Ridge Regression:** Configured with a default penalty coefficient ($\alpha = 1.0$) to add an L2 regularization penalty, minimizing weight inflation and ensuring stability.
- **Decision Tree Regressor:** Initially capped at a restricted tree depth ($max_depth = 5$) to isolate basic non-linear decision thresholds without introducing overfitting behaviors.

Optimization Milestone: Following baseline evaluation, an optimization sweep was conducted on the Decision Tree architecture. By increasing structural thresholds to $max_depth = 10$, the model successfully isolated deeper data patterns, capturing localized non-linear dependencies across the feature space and yielding a major jump in precision.

4. Performance Metrics and Comparative Evaluation

The evaluated models were checked against two primary tracking metrics: Root Mean Squared Error (RMSE), which represents the standard deviation of the residuals in original currency metrics, and the Coefficient of Determination (R^2 Score), tracking total variance explanation capabilities.

Evaluated Model Architecture	Root Mean Squared Error (RMSE)	Variance Explanation (R^2 Score)	Performance Status
Linear Regression (Baseline)	0.745581	0.575788	Underperforming Benchmark
Ridge Regression (Regularized Baseline)	0.745554	0.575819	Marginal Baseline Improvement
Decision Tree (Baseline, Depth = 5)	0.724234	0.599732	Moderate Non-Linear Capture
Decision Tree (Optimized, Depth = 10)	0.644362	0.683151	Production Selection (Optimal)

Performance Analysis:

The standard linear structures hit a definitive performance wall, explaining roughly 57.5% of dataset variance. This confirms that the relationships governing housing values across California coordinates contain deep non-linear attributes that traditional hyperplanes cannot map effectively. Transitioning to a Decision Tree architecture instantly unlocked better predictive capacity. At the optimized tier ($depth = 10$), the model achieved our peak validation score, minimizing standard error metrics down to **0.644362** and successfully explaining **68.3151%** of test distribution variance.

5. Serialization and Deployment Pipeline

To integrate the top-performing optimized decision tree model into production pipelines without wasting overhead on re-training routines, the active memory graph was fully serialized to disk using the [joblib](#) framework. This compiles the internal tree structures and split criteria directly into an optimized binary file format.

```
# Production Model Persistence Export Execution
import joblib

# Exporting optimized matrix structures for cold deployment
model_filename = "best_house_price_model.pkl"
joblib.dump(optimized_tree_model, model_filename)

# Verification pipeline instantiation for service assurance
production_model_engine = joblib.load(model_filename)
validation_outputs = production_model_engine.predict(X_test)
```

6. Visual Analysis

